

Single-Point VQE

The variational principle, ansatz design, and the first quantum result

Phase 3 runs the algorithm the project is named after — once, at one geometry, with a single success criterion: reproduce Phase 1's exact energy. Because Phase 2 already verified the Hamiltonian, any discrepancy here is unambiguously the fault of the ansatz or the optimiser, which is what makes the check diagnostic rather than merely reassuring.

Estimated time: **2-4 hours**.

In this document

The variational principle, and why an energy below FCI means a bug

The ansatz as the physics content of the algorithm — not an implementation detail

Hartree-Fock reference states, excitation operators, and UCCSD

Why converging on iteration count instead of energy quietly costs you accuracy

Full build guide with real output, and how to read every line of it

1. Where this phase sits

Everything so far has been setup. Phase 3 finally runs the quantum algorithm — at one fixed geometry, with one job: reproduce Phase 1's FCI energy to better than a millihartree. Phase 2's verification is what makes that check meaningful, because you already know the Hamiltonian is right.

What you will have at the end of Phase 3

A working VQE that converges to -1.13730604 Ha at 0.735 Å, matching FCI to roughly 4×10^{-6} mHa.

A parameterised ansatz built from actual chemistry — Hartree-Fock reference state plus excitation operators — rather than a generic circuit template.

A convergence criterion based on energy change rather than a fixed iteration count.

Two assertions that catch the two ways this can silently go wrong.

2. Theory

2.1 The variational principle

Everything rests on one theorem. For **any** normalised trial state $|\psi\rangle$ whatsoever:

$$\langle\psi|\hat{H}|\psi\rangle \geq E_0$$

where E_0 is the true ground-state energy. You can never accidentally go too low. Two consequences follow, and both are load-bearing:

- **"Lower is better" is always a valid search signal.** You do not need to know the answer to make progress towards it — you just minimise.
- **Your final number is an honest upper bound.** If VQE returns an energy *below* your FCI reference, you do not have a better answer; you have a bug. This is why Listing 3.2 asserts on it.

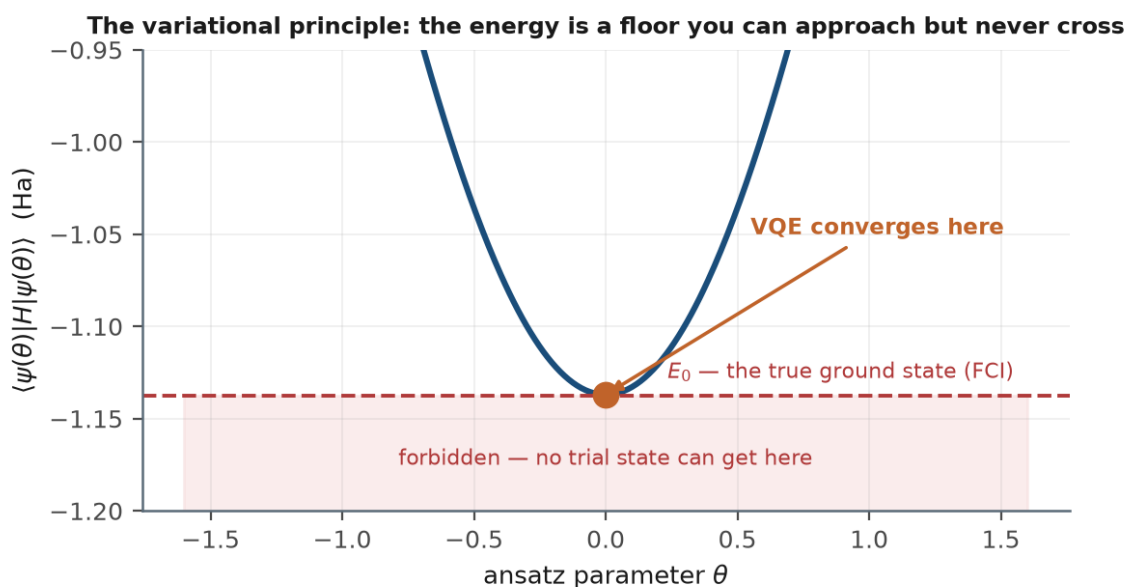


Figure 3.1 — The true ground-state energy is a floor. The shaded region is unreachable by any trial state.

2.2 The ansatz — where the physics lives

Ansatz German for "approach"; plural *ansätze*. The parameterised guess: a quantum circuit with adjustable dials. Turning the dials sweeps out a family of states, and VQE searches that family for the lowest energy.

Parameters The rotation angles θ . What the classical optimiser adjusts.

The consequence that matters most

VQE can only find the best state your ansatz is capable of expressing. If the true ground state is not in the family your circuit generates, the optimiser will converge confidently and smoothly onto the wrong answer, with no internal signal that anything is amiss.

Ansatz choice is therefore not a detail of implementation. It is the physics content of the algorithm, and it is the entire subject of Phase 6.

2.3 The reference state

Before any dial is turned, the circuit must be initialised. The natural choice is the Hartree-Fock occupation from Phase 1, expressed as a computational basis state — for H_2 that is $|1100\rangle$, meaning the lowest two spin orbitals are filled.

Starting from HF means VQE begins already ~99% correct and has only to find the correlation energy. That is a large practical advantage and it is also conceptually clean: the quantum circuit is being asked to supply exactly the thing the classical method could not.

Wire-ordering conventions differ between frameworks

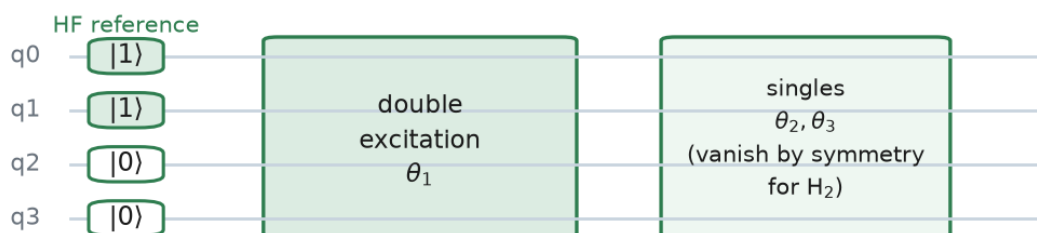
PennyLane writes computational basis states with **wire 0 leftmost**, so the H_2 Hartree-Fock state is $|1100\rangle$ and `qml.qchem.hf_state(2, 4)` returns the array `[1 1 0 0]`.

Qiskit reads Pauli strings and basis states **right-to-left**, so the same physical state is written $|0011\rangle$ there. Neither is wrong; mixing them silently is. If you are porting code between frameworks, check this first.

2.4 Excitations and UCCSD

Excitation	Moving electrons from occupied orbitals into empty ("virtual") ones. These are the building blocks of correlation.
Single	Moves one electron. For H_2 there are two: $[0 \rightarrow 2]$ and $[1 \rightarrow 3]$.
Double	Moves two electrons at once. For H_2 there is exactly one: $[0,1 \rightarrow 2,3]$.
UCCSD	U nitary C oupled C luster with S ingles and D oubles. An ansatz built directly from those excitation operators. Chemistry-motivated: its structure mirrors how electron correlation actually works.

UCCSD — chemistry-motivated, 3 parameters



Hardware-efficient (TwoLocal-style) — 24 parameters, no chemical structure



Figure 3.2 — UCCSD against the generic hardware-efficient template. Three meaningful parameters against twenty-four arbitrary ones.

For H_2 in a minimal basis, UCCSD is **provably exact** — it spans the same space as FCI. That is why Phase 3's target is not "close to FCI" but "equal to FCI within numerical precision".

Of the three parameters, only the double excitation does real work. The two singles vanish by symmetry: in a closed-shell molecule at a symmetric geometry there is nothing for a single excitation to contribute. You will see this directly in the output — two of the three optimal parameters converge to approximately zero.

2.5 Optimisers

Expectation value	$\langle \psi H \psi \rangle$, the average energy of a state. The quantity VQE minimises. On a simulator it is computed exactly; on hardware it is estimated by repeated measurement.
Gradient descent / Adam	Gradient-based methods. On quantum circuits gradients come from the parameter-shift rule , which gets exact derivatives by re-running the circuit at shifted parameter values. Fast when evaluations are clean.
COBYLA / Nelder-Mead	Gradient-free methods. Robust when energy evaluations are noisy, which is why hardware-oriented tutorials favour them. On a state-vector simulator you do not need them.
backprop	PennyLane's simulator can differentiate the circuit directly, which is much faster than parameter-shift and is the right choice here. Not available on real hardware.

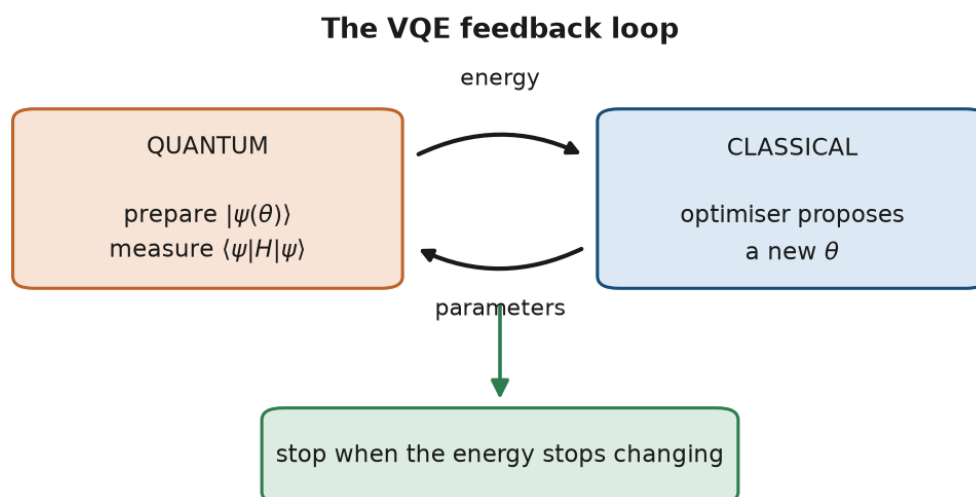
2.6 Converge on energy, not on iteration count

A fixed iteration cap is not a convergence criterion

`max_iter=50` answers the question "how long am I willing to wait?", not "has this converged?". Those are different questions, and only the second one has a right answer.

Stop when the **energy stops changing**: $|E_n - E_{n-1}| < \text{tol}$. Keep a maximum iteration count as a safety net, but make the energy criterion the one that normally fires.

Use a tolerance well below chemical accuracy. This project uses 10^{-9} Ha; a tolerance of 10^{-2} Ha is six orders of magnitude too coarse and will stop you far short of the answer while looking like success.



the variational principle guarantees $\langle H \rangle \geq E_0$ always — lower is always better

Figure 3.3 — The feedback loop. The quantum device supplies energies; the classical optimiser supplies parameters.

3. Build guide

Step 3.1 — Build the ansatz

Create `vqe_single.py`. The ansatz builder does three things: enumerate the excitations, convert them to circuit wires, and fetch the Hartree-Fock reference state.

Listing 3.1 — the ansatz builder

```
"""vqe_single.py - VQE at one fixed geometry, checked against FCI."""

import numpy as np
import pennylane as qml
from pennylane import numpy as npn

from classical import classical_energies
from hamiltonian import ELECTRONS, qubit_hamiltonian

def build_uccsd(n_qubits, electrons=ELECTRONS, verbose=False):
    """Return (ansatz_fn, n_params) for a UCCSD circuit.

    The ansatz starts from the Hartree-Fock state and applies every single
    and double excitation available in this orbital space.

    verbose is off by default because Phase 4 calls this inside a loop.
    """
    singles, doubles = qml.qchem.excitations(electrons, n_qubits)
    s_wires, d_wires = qml.qchem.excitations_to_wires(singles, doubles)
    hf = qml.qchem.hf_state(electrons, n_qubits)

    if verbose:
        print(f" reference state {hf}")
        print(f" single excitations {singles}")
        print(f" double excitations {doubles}")

    def ansatz(params, wires):
        qml.UCCSD(params, wires, s_wires=s_wires, d_wires=d_wires,
                  init_state=hf)

    return ansatz, len(singles) + len(doubles)
```

Step 3.2 — The optimisation loop

Listing 3.2 — the optimiser loop

```
def run_vqe(H, n_qubits, ansatz, params, stepsize=0.15,
            max_iter=400, tol=1e-9, verbose=False):
    """Minimise <psi(params)|H|psi(params)>.

    Converges on CHANGE IN ENERGY, not on a fixed iteration count.
    Returns (energy, params, history).
    """
    dev = qml.device("default.qubit", wires=n_qubits)

    @qml.qnode(dev, diff_method="backprop")
    def cost(p):
        ansatz(p, wires=range(n_qubits))
        return qml.expval(H)

    opt = qml.AdamOptimizer(stepsize=stepsize)
```

```

history = []
e_prev = np.inf

for i in range(max_iter):
    params, energy = opt.step_and_cost(cost, params)
    history.append(float(energy))
    if verbose and i % 20 == 0:
        print(f"    iter {i:4d}    E = {energy:.10f} Ha")
    if abs(energy - e_prev) < tol:
        break
    e_prev = energy

return float(cost(params)), params, history

```

Three details worth noting. `pnnp` is PennyLane's wrapped NumPy — parameters must be created with `requires_grad=True` so the framework knows to differentiate through them. `diff_method="backprop"` differentiates the simulator directly and is far faster than the parameter-shift rule. And the loop returns `cost(params)` rather than the last value from `step_and_cost`, because the latter is the energy *before* the final update.

Step 3.3 — Drive it and assert

Listing 3.3 — the driver, with the two assertions that matter

```

def main(r_angstrom=0.735):
    print(f"VQE for H2 at {r_angstrom} Angstrom\n")

    e_hf, e_fci, _ = classical_energies(r_angstrom)
    H, n_qubits = qubit_hamiltonian(r_angstrom)
    print(f"  qubits          {n_qubits}")

    ansatz, n_params = build_uccsd(n_qubits)
    print(f"  free parameters   {n_params}\n")

    params = pnnp.zeros(n_params, requires_grad=True)
    energy, params, history = run_vqe(H, n_qubits, ansatz, params,
                                     verbose=True)

    print()
    print(f"  Hartree-Fock      {e_hf:14.8f} Ha")
    print(f"  VQE               {energy:14.8f} Ha")
    print(f"  FCI (exact)       {e_fci:14.8f} Ha")
    print(f"  VQE error         {abs(energy - e_fci) * 1000:14.8f} mHa")
    print(f"  correlation found  "
          f"{{(e_hf - energy) / (e_hf - e_fci) * 100:13.4f} %}")
    print(f"  iterations        {len(history):14d}")
    print(f"  optimal parameters {np.round(params, 6)}")

    assert energy >= e_fci - 1e-9, "below FCI - variational principle violated"
    assert abs(energy - e_fci) < 1e-6, "VQE did not reach chemical accuracy"
    print("\n VQE verified against FCI")
    return energy

if __name__ == "__main__":
    main()

```

Step 3.4 — Run it

Actual output from the reference implementation

```

$ python vqe_single.py
VQE for H2 at 0.735 Angstrom

qubits          4
reference state  [1 1 0 0]
single excitations [[0, 2], [1, 3]]
double excitations [[0, 1, 2, 3]]
free parameters  3

iter   0   E = -1.1169989969 Ha
iter  20   E = -1.1363759059 Ha
iter  40   E = -1.1372967627 Ha
iter  60   E = -1.1372755379 Ha
iter  80   E = -1.1373017410 Ha
iter 100   E = -1.1373053614 Ha
iter 120   E = -1.1373060009 Ha

Hartree-Fock      -1.11699900 Ha
VQE               -1.13730603 Ha
FCI (exact)       -1.13730604 Ha
VQE error         0.00000389 mHa
correlation found 100.0000 %
iterations        131
optimal parameters [-8.10000e-05 -8.10000e-05  2.23559e-01]

VQE verified against FCI

```

Step 3.5 — Read the output properly

- **Iteration 0 is exactly the Hartree-Fock energy** (-1.1169989969). That is the reference state before any dial has moved, and seeing it confirms your initialisation is correct.
- **The final error is 3.9×10^{-6} mHa**, which is roughly four hundred thousand times better than chemical accuracy. UCCSD is exact for this system, so anything worse indicates an optimisation problem rather than an ansatz problem.
- **100.0000% of the correlation energy** was recovered. This is the headline number: the quantum algorithm captured the entire difference between mean-field theory and the exact answer.
- **Two of the three parameters are $\approx -8 \times 10^{-5}$** , i.e. zero to within optimiser noise. Those are the singles, vanishing by symmetry exactly as predicted. The one parameter doing real work is the double excitation, at 0.2236 radians.
- **131 iterations**, terminated by the energy criterion rather than the 400-iteration cap.

4. Traps in this phase

Trap	Symptom	Fix
Fixed iteration cap	Energy looks converged but is a few mHa high.	Converge on $ \Delta E < \text{tol}$ with $\text{tol} \approx 10^{-9}$ Ha. Keep <code>max_iter</code> only as a safety net.
Forgetting <code>requires_grad=True</code>	Gradients are zero; the energy never moves off the HF value.	Create parameters with <code>pnnp.zeros(n, requires_grad=True)</code> , not plain <code>np.zeros</code> .
Energy below FCI	VQE returns something lower than your reference.	The variational principle forbids it. Either the Hamiltonians differ (check Phase 2 again) or you are comparing an electronic-only energy against a total energy.
Wrong reference state	Converges, but to a visibly wrong value, or converges very slowly.	Use <code>qml.qchem.hf_state(electrons, qubits)</code> . Do not hand-write the bit string — the wire-ordering convention will catch you out.
Random initialisation	Different answers on different runs.	Start from zeros. With a HF reference state, zeros mean "the Hartree-Fock state", which is an excellent starting point. Random angles throw that advantage away.
Stepsize too large	Energy oscillates or diverges.	Adam at 0.15 works well here. If you see oscillation, halve it.

5. What was done in this phase

- **Ran the quantum algorithm** for the first time, and verified it against a reference derived by a completely independent classical method.
- **Established the variational principle** as both the search signal and the correctness check — an energy below FCI is a bug, not a discovery.
- **Built a chemistry-motivated ansatz**: Hartree-Fock reference state, excitation operators enumerated from the orbital space, UCCSD applied on top. Three parameters, of which one matters.
- **Confirmed the singles vanish by symmetry**, which is a prediction of the theory that the output visibly bears out.
- **Recovered 100% of the correlation energy** — the entire gap between Hartree-Fock and exact — to an error of 4×10^{-6} mHa.
- **Converged on energy change rather than iteration count**, and learned why the difference is not pedantic.
- **Encountered the wire-ordering convention** that differs between PennyLane and Qiskit, which is the sort of thing that silently corrupts ported code.

6. Checklist before moving on

1. `vqe_single.py` converges to -1.1373060 Ha at $0.735 \text{ \AA}$.
2. Both assertions pass.

3. Iteration 0 of the optimiser equals the Hartree-Fock energy.
4. Two of the three optimal parameters are approximately zero, and you can say why.
5. You have run the same script at a second geometry — try 1.5 Å — and it still matches FCI.
6. You can state what would happen if your ansatz could not express the true ground state, and why the optimiser would not warn you.

Next

Phase 4 — The geometry scan. You have one point. The project needs forty. Two implementation details separate a clean run from a frustrating one, and both are easy to get wrong.